Project Orange – EURO/ROADEF Challenge 2026 “Keep The Flow!”

Mô hình quy hoạch tuyến tính nguyên cho bài toán T-Adaptive Segment Routing

Họ và tên

25/10/2026

1 Giới thiệu

Bối cảnh. Orange vận hành nhiều mạng backbone IP/MPLS quy mô quốc gia và quốc tế, với hàng nghìn router, hàng nghìn liên kết và hàng chục nghìn luồng lưu lượng (*demand*) mỗi năm. Mục tiêu của kỹ thuật lưu lượng (*Traffic Engineering*) là dùng hạ tầng sẵn có hiệu quả hơn, chủ yếu để tránh nghẽn và giữ chất lượng dịch vụ. Theo quy tắc thiết kế chuẩn, mạng phải tiếp tục hoạt động khi một liên kết hay một nút không còn dùng được, kể cả khi điều đó là do nhà vận hành chủ động tắt để bảo trì.

Động lực. Theo cách truyền thống, gói tin đi theo đường ngắn nhất tính theo trọng số ω do người quản trị gán cho các liên kết. Cách này dễ triển khai, nhưng nhiều luồng thường dồn vào cùng vài liên kết, trong khi muốn điều chỉnh thì phải tìm lại cả bộ trọng số, một bài toán rất khó [5]. *Segment Routing* (SR) [4] khắc phục điều này: tại nút nguồn, một danh sách nút trung gian (*waypoint*) được ghi vào header của gói tin. Gói tin đi theo đường ngắn nhất tới waypoint thứ nhất, rồi tới waypoint tiếp theo, và cứ thế cho tới đích. Chỉ cần chọn waypoint là có thể lái lưu lượng tránh các liên kết nghẽn mà không phải sửa trọng số.

Khi một liên kết bị tắt để bảo trì, các đường ngắn nhất thay đổi và tình trạng nghẽn có thể xuất hiện ở chỗ khác. Ta có thể đổi waypoint giữa các chu kỳ, nhưng mỗi lần đổi là một thao tác vận hành tốn kém, nên tổng số thay đổi bị giới hạn bởi một *budget*.

Mô tả bài toán. Bài toán *T-Adaptive Segment Routing* (T-ASR) [1, 2] có dữ liệu vào gồm:

- một mạng có hướng với trọng số và dung lượng trên mỗi cung;
- một tập demand, mỗi demand có lưu lượng thay đổi theo chu kỳ thời gian;
- một lịch bảo trì cho biết những cung nào bị tắt ở mỗi chu kỳ;
- số segment tối đa của một đường;
- budget thay đổi giữa hai chu kỳ liên tiếp.

Yêu cầu là chọn, cho mỗi demand và mỗi chu kỳ, một danh sách waypoint sao cho tải của các liên kết là nhỏ nhất. Đề gốc dùng mục tiêu lexicographic trên toàn bộ vector tải. Báo cáo này dùng thành phần đầu tiên và quan trọng nhất của mục tiêu đó: *Maximum Link Utilization* (MLU). Bài toán là NP-khó và khó xấp xỉ, ngay cả trên topology rất đơn giản [6].

Nội dung báo cáo. Mục 2 trình bày mô hình MILP: dữ liệu, cách tính hệ số chia luồng, biến, mục tiêu và ràng buộc. Mục 3 trình bày thí nghiệm trên 20 instance SetA, trước tiên chỉ với chu kỳ $t = 0$ (giới hạn 1800 s), sau đó với hai chu kỳ $t \in \{0, 1\}$ có ràng buộc budget (giới hạn 3600 s).

2 Mô hình toán học

2.1 Dữ liệu

Bảng 1 liệt kê các tham số và nguồn lấy chúng. Mỗi instance gồm ba file JSON: `net` (mạng), `tm` (ma trận lưu lượng) và `scenario` (kịch bản bảo trì).

Ký hiệu	Ý nghĩa	Nguồn
$G = (V, A)$	đồ thị có hướng, $n = \|V\|$ nút, $\|A\|$ cung	`net`: `nodes`, `links`
$\omega(a)$	trọng số (metric) của cung a, chỉ dùng để tính đường ngắn nhất	`net`: `metric`
$c(a)$	dung lượng của cung a	`net`: `capacity`
D, (s_d, t_d)	tập demand, nguồn và đích của demand d	`tm`: `s`, `t`
$\nu(d, t)$	lưu lượng của demand d tại chu kỳ t	`tm`: `v[t]`
$T = \{0, \ldots, h-1\}$	các chu kỳ, $T^* = T \setminus \{0\}$	`tm`: `num_time_slots`
$maxSeg$	số segment tối đa của một đường	`scenario`: `max_segments`
$q(t)$	tập cung bị tắt tại chu kỳ $t \in T^*$; $G_t = (V, A \setminus q(t))$	`scenario`: `interventions`
$\kappa(t)$	số thay đổi tối đa giữa chu kỳ $t-1$ và t	`scenario`: `budget`
$r(i, j, a, t)$	tỉ lệ luồng từ i đến j đi qua cung a trong G_t	**tính trước** (mục 2.2)

Bảng 1: Tham số của mô hình.

Tại $t = 0$ không có cung nào bị tắt và budget là vô hạn, nên chu kỳ 0 không xuất hiện trong file `scenario`. ID trong file JSON không nhất thiết theo thứ tự (ví dụ setA-14 liệt kê nút từ 249 về 0), nên chương trình tạo một bảng ánh xạ từ ID sang chỉ số $1..n$.

Segment path và segment. Một *segment path* $\langle s, w_1, \ldots, w_\ell, t\rangle$ gồm nguồn s, các waypoint $w_1, \ldots, w_\ell$ và đích t. Mỗi cặp nút liên tiếp $(s, w_1), (w_1, w_2), \ldots, (w_\ell, t)$ là một *segment*. Luồng trên segment (i, j) đi theo các đường ngắn nhất từ i đến j, tức theo *forwarding graph* $FG(i, j)$. Khi có nhiều đường ngắn nhất, cơ chế ECMP chia đều luồng *tại mỗi nút* cho các cung ra của nút đó trong $FG(i, j)$.

2.2 Tính hệ số chia luồng $r(i, j, a, t)$

Hệ số $r(i, j, a, t)$ chỉ phụ thuộc vào G_t và ω, không phụ thuộc vào quyết định chọn waypoint, nên được tính trước như dữ liệu. Nhờ vậy tải là hàm *tuyến tính* của biến quyết định. Phải tính r cho *mọi* cặp (i, j), vì nút nào cũng có thể là waypoint. Cách tính như sau:

1. Chạy Dijkstra từ mọi nút để có d_{ij}, độ dài đường ngắn nhất từ i đến j trong G_t. Chương trình dùng `dijkstra_shortest_paths` của Graphs.jl.
2. Cung $a = (u, v)$ nằm trên một đường ngắn nhất từ i đến j khi và chỉ khi $d_{iu} + \omega(a) + d_{vj} = d_{ij}$. Ở bốn instance có trọng số là số thực (05, 08, 12, 14), đẳng thức này phải được kiểm tra với sai số tương đối 10^{-9}. Cùng một độ dài nhưng cộng theo thứ tự khác nhau có thể lệch nhau ở chữ số cuối; nếu so sánh bằng `==`, hơn một nửa số cặp nút của setA-08, 12 và 14 bị "mất" luồng.
3. Với mỗi đích j, xét các nút theo thứ tự $d_{\cdot j}$ giảm dần, đây là một thứ tự topo vì $\omega > 0$. Đặt một đơn vị luồng tại i. Tại mỗi nút u, chia đều lượng luồng đang có cho các cung ra (u, v) thoả $d_{uj} = \omega(u, v) + d_{vj}$. Lượng luồng đi qua cung a chính là $r(i, j, a, t)$.

So sánh với cách đếm đường ngắn nhất. Tài liệu hướng dẫn `calculating_r.pdf` đề xuất công thức $r_{ija} = \sigma_{iu}\sigma_{vj}/\sigma_{ij}$, trong đó σ là *số* đường ngắn nhất. Công thức này chia luồng đều cho từng *đường*, còn ECMP trong đề chia đều tại từng *nút*. Hai cách cho kết quả khác nhau khi các đường ngắn nhất tách ra rồi nhập lại. Ví dụ, với demand (v_0, v_5) của instance toy có ba đường ngắn nhất:

- theo ECMP: $r(a_{1,3}) = r(a_{1,4}) = 0.5$ và $r(a_{3,5}) = 0.75$, đúng như Table 2 của đề;
- theo cách đếm đường: $r(a_{1,3}) = 1/3$, $r(a_{1,4}) = 2/3$ và $r(a_{3,5}) = 2/3$.

Chúng tôi cài cả hai cách và dùng ECMP cho mô hình, vì cách này khớp với định nghĩa và ví dụ của đề. Trên SetA, ở cả hai chu kỳ, hai cách cho cùng giá trị ở mọi cặp nút, trừ một cặp ở setA-18 và một cặp ở setA-20 (không phải cặp nguồn–đích của demand nào). Lý do là trong SetA, đường ngắn nhất hầu như luôn duy nhất: ở mỗi instance, không quá 1% số cặp nút có nhiều hơn một đường ngắn nhất.

2.3 Biến quyết định

Coi mỗi cặp nút (i, j), $i \neq j$, là một "cung ảo" của một đồ thị đầy đủ H trên V. Khi đó một segment path cho demand d chính là một đường đi từ s_d đến t_d trong H.

- $x_{ij}^{dt} \in \{0, 1\}$: bằng 1 nếu segment path của demand d tại chu kỳ t chứa segment (i, j). Chỉ tạo biến cho các cặp (i, j) có đường đi từ i tới j trong G_t; các cặp khác không dùng được, nên coi $x_{ij}^{dt} = 0$.
- $U \geq 0$: MLU, tức tải lớn nhất trên mọi cung và mọi chu kỳ.
- $y_{ij}^{dt} \geq 0$ (chỉ khi có budget): biến phụ để tuyến tính hoá $|x_{ij}^{dt} - x_{ij}^{d,t-1}|$.

2.4 Hàm mục tiêu

$$\min\ U \qquad \text{với} \qquad \lambda(a, t) \leq U \quad \forall t \in T,\ a \in A \setminus q(t).$$

Tải $\lambda(a, t)$ là tỉ số giữa lưu lượng đi qua cung a và dung lượng $c(a)$. Mục tiêu lexicographic của đề cực tiểu tải lớn nhất, rồi tải lớn thứ hai, và cứ thế tiếp tục. Thành phần đầu tiên của nó chính là $\min U$, nên mọi nghiệm tối ưu lexicographic đều tối ưu theo MLU [2]. Điều ngược lại không đúng: hai nghiệm có cùng MLU có thể khác nhau ở các tải nhỏ hơn.

2.5 Ràng buộc

(1) Bảo toàn luồng. Mỗi demand đi trên đúng một segment path từ nguồn tới đích:

$$\sum_{j \in V \setminus \{i\}} x_{ij}^{dt} - \sum_{j \in V \setminus \{i\}} x_{ji}^{dt} = \begin{cases} 1 & i = s_d \\ -1 & i = t_d \\ 0 & \text{khác} \end{cases} \qquad \forall i \in V,\ d \in D,\ t \in T. \tag{1}$$

Tại nguồn, số segment đi ra nhiều hơn số đi vào đúng một; tại đích thì ngược lại; tại mỗi waypoint, số segment đi vào bằng số đi ra.

(2) Số segment. Giới hạn độ dài header, tính cả segment đầu (từ nguồn) và segment cuối (tới đích):

$$\sum_{i,j \in V} x_{ij}^{dt} \leq maxSeg \qquad \forall d \in D,\ t \in T. \tag{2}$$

(3) Tải. Lưu lượng trên cung a là tổng, trên mọi demand và mọi segment được chọn, của phần luồng đi qua a:

$$\sum_{d \in D} \sum_{i,j \in V} r(i, j, a, t)\, \nu(d, t)\, x_{ij}^{dt} \ \leq\ U\, c(a) \qquad \forall t \in T,\ a \in A \setminus q(t). \tag{3}$$

Vì r và ν là hằng số nên ràng buộc này tuyến tính. Các cung trong $q(t)$ có $r = 0$ vì r được tính trên G_t.

(4) Budget. Số segment bị thêm hoặc bị bỏ giữa hai chu kỳ liên tiếp không vượt quá $\kappa(t)$:

$$\sum_{d \in D} \sum_{i \neq j} \left| x_{ij}^{dt} - x_{ij}^{d,t-1} \right| \leq \kappa(t) \qquad \forall t \in T^*. \tag{4}$$

Trị tuyệt đối không tuyến tính, nên ta thay (4) bằng

$$y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1}, \qquad y_{ij}^{dt} \geq x_{ij}^{d,t-1} - x_{ij}^{dt}, \qquad \sum_{d} \sum_{i \neq j} y_{ij}^{dt} \leq \kappa(t). \tag{4′}$$

Hai bất đẳng thức đầu buộc $y_{ij}^{dt} \geq |x_{ij}^{dt} - x_{ij}^{d,t-1}|$, nên ràng buộc thứ ba kéo theo (4). Ngược lại, mọi nghiệm thoả (4) đều mở rộng được thành một nghiệm của hệ trên bằng cách đặt y bằng đúng trị tuyệt đối. Segment nào có đường đi ở chu kỳ này mà không có ở chu kỳ kia thì biến tương ứng được coi là 0. Ví dụ, đổi $\langle s, w, t\rangle$ thành $\langle s, t\rangle$ tốn 3 đơn vị: bỏ (s, w) và (w, t), thêm (s, t). Mỗi demand đổi đường tốn ít nhất 2 đơn vị, vì hiệu $x^t - x^{t-1}$ là một luồng vòng khác 0. Do đó $\kappa(t) = 1$ nghĩa là không demand nào được đổi đường.

Chu trình rời. Ràng buộc (1) không cấm x^{dt} chứa thêm một chu trình không nối với đường đi từ s_d tới t_d, ví dụ $x_{ij}^{dt} = x_{ji}^{dt} = 1$. Chu trình như vậy không thuộc segment path nào. Với một chu kỳ, nó chỉ làm tải tăng nên không ảnh hưởng tới giá trị tối ưu. Khi có budget, nó có thể làm vế trái của (4) nhỏ hơn số thay đổi thật $dist(P^{t-1}, P^t)$ theo định nghĩa của đề. Vì vậy, sau khi giải, chúng tôi đọc segment path bằng đường đi Euler từ s_d, bỏ các chu trình rời, rồi tính lại tải và $dist$ từ các path này (mục 3.4).

Tóm tắt. Mô hình cho chu kỳ $t = 0$ (Step 5) gồm mục tiêu $\min U$ với các ràng buộc (1)–(3) và $x \in \{0, 1\}$. Mô hình cho hai chu kỳ (Step 8) gồm (1)–(3) cho mỗi $t \in \{0, 1\}$, cộng thêm (4) đã tuyến tính hoá.

2.6 Ví dụ kiểm tra: instance toy

Instance toy của đề có 7 nút, 22 cung và 2 demand, (v_0, v_5) và (v_2, v_5), với $maxSeg = 4$. Tại $t = 1$ các cung $a_{1,3}$ và $a_{1,4}$ bị tắt, và $\kappa(1) = 20$.

- Khi không có waypoint, MLU tại $t = 0$ là 0.4375, đạt trên cung $a_{3,5}$.
- Mô hình Step 5 trả về MLU tối ưu 0.375, chẳng hạn với segment path $\langle v_0, v_4, v_5\rangle$ như nghiệm minh hoạ trong Figure 4 của đề.
- Tại $t = 1$, routing mặc định đã có MLU 0.375 (các cung $a_{2,4}, a_{4,3}, a_{4,6}, a_{3,5}, a_{6,5}$ cùng ở mức này) và không giảm được nữa. Budget $\kappa(1) = 20$ không ràng buộc, vì hai demand đổi đường tốn tối đa $2 \cdot (4 + 4) = 16$ đơn vị. Do đó giá trị tối ưu của mô hình Step 8 là 0.375, đúng như kết quả của solver.

Các giá trị tối ưu trên đã được xác nhận bằng cách vét cạn mọi segment path có tối đa $maxSeg$ segment (206 path cho mỗi demand ở mỗi chu kỳ). Nghiệm tối ưu không duy nhất, nên solver có thể trả về waypoint khác nhau tuỳ máy và seed.

2.7 Kích thước mô hình

Số biến nhị phân cho mỗi chu kỳ xấp xỉ $|D| \cdot n(n-1)$. Ràng buộc (3) có khoảng $|D| \cdot \text{nnz}(r)$ hệ số khác 0, với $\text{nnz}(r)$ là số cặp (segment, cung) có $r > 0$. Trên SetA, số biến nhị phân đi từ $1.5 \cdot 10^4$ (setA-01) tới $9.6 \cdot 10^8$ (setA-20) cho mỗi chu kỳ (Bảng 2). Đây là lý do mô hình chỉ dựng được cho các instance nhỏ trên máy thí nghiệm (mục 3).

3 Thí nghiệm số

3.1 Mô tả instance

SetA gồm 20 instance (Bảng 2). Mọi instance đều có $h = 2$ chu kỳ, $maxSeg = 6$, và đúng một cung bị tắt tại $t = 1$. Với mọi cung $u \to v$ đều có cung ngược $v \to u$, và mỗi cặp nguồn–đích chỉ có một demand. Mười instance có cờ `multigraph` trong file JSON (05, 06, 08, 10, 12, 13, 14, 16, 17, 19), nhưng không instance nào có cung song song. Trọng số ω là số nguyên ở 16 instance và là số thực ở 05, 08, 12, 14 (ví dụ 0.0037). Cột MLU_0 trong các bảng kết quả là MLU khi mọi demand đi thẳng theo đường ngắn nhất (không waypoint): tính tại $t = 0$ ở Bảng 3, và lấy max trên hai chu kỳ ở Bảng 5. Giá trị này xấp xỉ 1 ở đa số instance, tức routing mặc định làm liên kết nặng nhất đầy đúng 100%.

3.2 Môi trường và thiết lập

- **Máy**: Intel Core 7 240H (10 nhân: 6 P-core và 4 E-core, 16 luồng), 16 GB RAM, Linux (WSL2, kernel 6.6).

Inst	$\|V\|$	$\|A\|$	$\|D\|$	$\kappa(1)$	#biến x (t=0)	Inst	$\|V\|$	$\|A\|$	$\|D\|$	$\kappa(1)$	#biến x (t=0)
01	20	80	40	51	$1.5 \cdot 10^4$	11	200	1000	400	89	$1.6 \cdot 10^7$
02	30	150	45	63	$3.9 \cdot 10^4$	12	200	898	400	13	$1.6 \cdot 10^7$
03	50	250	20	53	$4.9 \cdot 10^4$	13	200	1000	2000	12	$8.0 \cdot 10^7$
04	50	250	200	44	$4.9 \cdot 10^5$	14	250	1108	600	13	$3.7 \cdot 10^7$
05	100	396	100	1	$9.9 \cdot 10^5$	15	250	1250	600	54	$3.7 \cdot 10^7$
06	100	500	500	13	$4.95 \cdot 10^6$	16	250	1452	4800	13	$3.0 \cdot 10^8$
07	100	500	800	90	$7.9 \cdot 10^6$	17	300	1270	2000	1	$1.8 \cdot 10^8$
08	150	654	200	13	$4.5 \cdot 10^6$	18	300	1500	2000	89	$1.8 \cdot 10^8$
09	150	750	200	18	$4.5 \cdot 10^6$	19	300	1998	6000	13	$5.4 \cdot 10^8$
10	150	966	1000	1	$2.2 \cdot 10^7$	20	400	2000	6000	90	$9.6 \cdot 10^8$

Bảng 2: Các instance SetA. #biến x là số biến nhị phân của mô hình Step 5.

- **Phần mềm**: Julia 1.13.2, JuMP 1.32.1, Graphs.jl 1.15.0, solver HiGHS 1.15.1 (HiGHS.jl 1.26.1).
- **Giới hạn thời gian**: 1800 s cho Step 6 và 3600 s cho Step 9. *CPU time* là thời gian của riêng lời gọi `optimize!`, lấy bằng `solve_time`.
- **Thiết lập solver**: tham số `mip_heuristic_effort` được đặt bằng 0.3 (mặc định là 0.05), vì thử nghiệm cho thấy cận dưới LP gần như luôn chặt, còn việc tìm nghiệm khả thi mới là phần chậm. Solver được cho một nghiệm khởi đầu là "không waypoint". Nghiệm này luôn khả thi, nên mọi lần chạy đều có cận trên.
- **Gap**: $1 - LB/UB$, với LB là cận dưới tốt nhất (`objective_bound`) và UB là giá trị nghiệm tốt nhất.
- **Số lần chạy**: đề yêu cầu báo cáo trung bình của 10 lần chạy. HiGHS là deterministic, nên mỗi lần chạy dùng một `random_seed` khác $(0, 1, \ldots, 9)$.
  - Một instance được chạy lại với seed tiếp theo chừng nào lần chạy vừa xong kết thúc Optimal trong dưới 300 s, tối đa 10 lần.
  - Gặp một lần chạy chậm hơn hoặc chạm giới hạn thì dừng; lần chạy đó vẫn được tính vào trung bình. Lý do là 10 lần $\times$ 1800–3600 s cho mỗi instance khó vượt quá thời gian có được.
  - Cột "Lần" ghi số lần chạy thực tế; các cột Obj, CPU, Gap và #nút là trung bình trên các lần đó.
- **Giới hạn bộ nhớ**: mô hình có hơn $5 \cdot 10^6$ biến không được dựng, vì riêng việc dựng mô hình $4.5 \cdot 10^6$ biến trong JuMP đã tốn 3.3 GB RAM, và HiGHS cần thêm một bản sao khi giải.
- **Chốt chặn cứng**: với mô hình lớn, HiGHS có thể chạy quá giới hạn thời gian (Step 9, setA-04: 9905 s so với giới hạn 3600 s), vì nó chỉ kiểm tra giới hạn ở một số thời điểm trong lúc presolve và giải LP gốc. Vì vậy mỗi instance (gồm mọi lần chạy của nó) chạy trong một tiến trình riêng, và tiến trình bị dừng nếu chạy quá 10800 s ở Step 9 hoặc 2700 s ở Step 6. Ở Step 6, các instance 01–05 đều kết thúc trước mức này. Instance bị dừng được ghi "Bị dừng", với Obj là MLU của nghiệm khởi đầu.

3.3 Kết quả với chu kỳ $t = 0$ (Step 6)

Bảng 3 ghi, cho mỗi instance:

- MLU khi không có waypoint (MLU_0);
- giá trị tốt nhất tìm được (Obj);
- CPU time trung bình (s), trạng thái và gap;
- số nút trung bình của cây branch-and-bound, và số lần chạy.

Inst	$\|V\|$	$\|A\|$	$\|D\|$	MLU_0	Obj	CPU (s)	Status	Gap	#nút	Lần
01	20	80	40	1.0000	0.9294	4.5	Optimal	0	1	10
02	30	150	45	1.0000	0.5486	318.0	Optimal	0	0	1
03	50	250	20	1.0000	0.9435	58.2	Optimal	0	1	9
04	50	250	200	0.6918	0.6918	1800.3	Not Opt.	33.64%	0	1
05	100	396	100	0.3333	0.1155	1706.0	Optimal	0	1	1
07	100	500	800	0.9080	*mô hình quá lớn (7.9 triệu biến)*					
10	150	966	1000	1.0000	*mô hình quá lớn (22.4 triệu biến)*					
11	200	1000	400	1.0000	*mô hình quá lớn (15.9 triệu biến)*					
12	200	898	400	1.0001	*mô hình quá lớn (15.9 triệu biến)*					
13	200	1000	2000	1.0000	*mô hình quá lớn (79.6 triệu biến)*					
14	250	1108	600	1.0000	*mô hình quá lớn (37.4 triệu biến)*					
15	250	1250	600	1.0000	*mô hình quá lớn (37.4 triệu biến)*					
16	250	1452	4800	0.9672	*mô hình quá lớn (298.8 triệu biến)*					
17	300	1270	2000	1.0001	*mô hình quá lớn (179.4 triệu biến)*					
18	300	1500	2000	0.6334	*mô hình quá lớn (179.4 triệu biến)*					
19	300	1998	6000	1.0000	*mô hình quá lớn (538.2 triệu biến)*					
20	400	2000	6000	1.0000	*mô hình quá lớn (957.6 triệu biến)*					

* CPU time vượt giới hạn: HiGHS không dừng được trong lúc presolve hoặc giải LP gốc. "–" ở cột Gap: không có cận dưới. "Bị dừng": vượt chốt chặn cứng, Obj là MLU của nghiệm khởi đầu.

Bảng 3: Kết quả Step 6: chỉ chu kỳ $t = 0$, giới hạn 1800 s.

Nhận xét.

- Mô hình giải tối ưu bốn instance 01, 02, 03 và 05. So với định tuyến theo đường ngắn nhất, Segment Routing giảm MLU 7.1% ở setA-01, 45.1% ở setA-02, 5.6% ở setA-03 và 65.3% ở setA-05.
- Ở mọi seed, giá trị tối ưu là như nhau; chỉ thời gian giải thay đổi, và thay đổi mạnh (setA-01: trung bình 4.5 s, độ lệch chuẩn 8.9 s; setA-03: 58 s và 93 s). Thời gian cũng không tăng đều theo kích thước: setA-02 (39 nghìn biến) mất 318 s, lâu hơn setA-03 (49 nghìn biến).
- Cây branch-and-bound chỉ có 0 hoặc 1 nút: cận dưới tại nút gốc (LP relaxation cộng các lát cắt của HiGHS) đã bằng giá trị tối ưu. Thời gian chủ yếu dành cho việc giải LP gốc và tìm một nghiệm nguyên đạt cận đó, nên tăng `mip_heuristic_effort` có ích.
- Với setA-04 (200 demand), solver không cải thiện được nghiệm khởi đầu trong 1800 s (gap 33.6%). Kết hợp với kết quả của biến thể ở Bảng 4, giá trị tối ưu của setA-04 nằm giữa 0.4591 và 0.4791.
- **[TODO: nhận xét setA-06, 08, 09 sau khi chạy xong]**
- Mười hai instance còn lại (07, 10–20) cần từ 7.9 triệu tới 958 triệu biến nhị phân, nên không dựng được mô hình trên máy 16 GB.

Biến thể không đi vòng. Với một chu kỳ, có thể thêm ràng buộc mỗi segment path đi qua mỗi nút tối đa một lần: $\sum_j x_{ji}^{d0} \le 1$ với mọi $i \ne s_d$, và không có segment đi vào s_d hay đi ra từ t_d. Ràng buộc này không làm thay đổi giá trị tối ưu, vì bỏ một vòng lặp chỉ làm tải giảm ($r \ge 0$). Ở Step 9 thì không dùng được, vì khi có budget, đường có vòng lặp có thể bắt buộc phải có trong nghiệm tối ưu [2]. Bảng 4 so sánh hai mô hình trên 01–05; biến thể này được chạy trên cùng máy (trong Docker), với cùng thiết lập.

Ràng buộc này làm 01–03 nhanh hơn 4 tới 19 lần, và tìm được nghiệm tốt hơn hẳn cho setA-04 (0.4791 so với 0.6918) với cùng cận dưới 0.4591. Ngược lại, với setA-05, HiGHS không giải xong LP gốc: sau 6037 s vẫn chưa có cận dưới, trong khi mô hình của đề giải tối ưu trong 1706 s. Vì kết quả không đồng nhất như vậy, Bảng 3 dùng đúng mô hình của đề.

Inst	Mô hình (1)–(3) của đề				Thêm ràng buộc không đi vòng			
	Obj	CPU (s)	Gap	Lần	Obj	CPU (s)	Gap	Lần
01	0.9294	4.5	0	10	0.9294	0.6	0	10
02	0.5486	318.0	0	1	0.5486	16.4	0	10
03	0.9435	58.2	0	9	0.9435	13.6	0	10
04	0.6918	1800.3	33.64%	1	0.4791	1800.4	4.18%	1
05	0.1155	1706.0	0	1	0.3333	6036.6*	–	1

Bảng 4: Step 6 trên 01–05 với hai mô hình. *: CPU time vượt giới hạn 1800 s; "–": không có cận dưới.

3.4 Kết quả với hai chu kỳ $t \in \{0, 1\}$ và budget (Step 9)

Bảng 5 có cùng các cột như Bảng 3; Obj là MLU lớn nhất trên cả hai chu kỳ, và MLU_0 là max của MLU không waypoint tại $t = 0$ và $t = 1$.

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	MLU_0	Obj	CPU (s)	Status	Gap	#nút	Lần
01	20	80	40	1.0000	0.9294	163.9	Optimal	0	1	5
02	30	150	45	1.0981	0.9031	172.4	Optimal	0	1	4
03	50	250	20	1.0000	0.9435	704.2	Optimal	0	1	1
04	50	250	200	1.0000	1.0000	9905.0*	Not Opt.	–	0	1
05	100	396	100	1.0000	1.0000	>10800	Bị dừng	–	–	1
06	100	500	500	1.0000	*mô hình quá lớn (14.8 triệu biến)*					
07	100	500	800	1.0000	*mô hình quá lớn (23.8 triệu biến)*					
08	150	654	200	1.0000	*mô hình quá lớn (13.4 triệu biến)*					
09	150	750	200	1.0000	*mô hình quá lớn (13.4 triệu biến)*					
10	150	966	1000	1.0000	*mô hình quá lớn (67.0 triệu biến)*					
11	200	1000	400	1.0000	*mô hình quá lớn (47.8 triệu biến)*					
12	200	898	400	1.0001	*mô hình quá lớn (47.8 triệu biến)*					
13	200	1000	2000	1.0000	*mô hình quá lớn (238.8 triệu biến)*					
14	250	1108	600	1.0000	*mô hình quá lớn (112.0 triệu biến)*					
15	250	1250	600	1.0000	*mô hình quá lớn (112.0 triệu biến)*					
16	250	1452	4800	1.0000	*mô hình quá lớn (896.4 triệu biến)*					
17	300	1270	2000	1.0001	*mô hình quá lớn (538.2 triệu biến)*					
18	300	1500	2000	1.0000	*mô hình quá lớn (538.2 triệu biến)*					
19	300	1998	6000	1.0000	*mô hình quá lớn (1614.6 triệu biến)*					
20	400	2000	6000	1.0000	*mô hình quá lớn (2872.8 triệu biến)*					

* CPU time vượt giới hạn: HiGHS không dừng được trong lúc presolve hoặc giải LP gốc. "–" ở cột Gap: không có cận dưới. "Bị dừng": vượt chốt chặn cứng, Obj là MLU của nghiệm khởi đầu.

Bảng 5: Kết quả Step 9: hai chu kỳ $t \in \{0, 1\}$ có ràng buộc budget, giới hạn 3600 s. Obj là MLU trên cả hai chu kỳ.

Nhận xét.

- Mô hình hai chu kỳ có khoảng gấp ba số biến của mô hình một chu kỳ (biến x cho hai chu kỳ và biến y), nên chỉ dựng được năm instance 01–05. Mô hình giải tối ưu 01, 02 và 03.
- Với setA-01 và 03, MLU hai chu kỳ bằng MLU tối ưu của riêng $t = 0$ (0.9294 và 0.9435). Giải riêng chu kỳ $t = 1$ (không có budget) cho MLU thấp hơn nhiều (0.3464 và 0.4026), nên ở hai instance này chu kỳ $t = 0$ là nút cổ chai.
- Với setA-02, định tuyến mặc định tại $t = 1$ làm một cung quá tải 109.8%. Mô hình đưa MLU xuống 0.9031, đúng bằng MLU tối ưu khi giải riêng $t = 1$ mà không có budget. Như vậy giới hạn nằm ở chính chu kỳ $t = 1$ (mạng thiếu một cung, với lưu lượng của $t = 1$), chứ không phải ở budget $\kappa(1) = 63$.
- Mục tiêu $\min U$ chỉ quan tâm chu kỳ nặng nhất. Trong nghiệm của setA-02 ở phần kiểm tra dưới đây,

MLU tại $t = 0$ là 0.8262, cao hơn hẳn mức 0.5486 đạt được khi chỉ xét $t = 0$. Mục tiêu lexicographic của đề sẽ tiếp tục cực tiểu các tải này.

- Với setA-04 (1.47 triệu biến), HiGHS không giải xong LP gốc: sau 9905 s, quá giới hạn 3600 s, vẫn chưa có cận dưới, và nghiệm tốt nhất vẫn là nghiệm khởi đầu. setA-05 (2.97 triệu biến, $\kappa(1) = 1$ nên không demand nào được đổi đường) bị dừng sau 10800 s mà chưa kết thúc.
- Mười lăm instance 06–20 cần từ 13.4 triệu tới 2.87 tỉ biến, nên không dựng được.

Kiểm tra nghiệm. Vì mô hình cho phép chu trình rời (mục 2), chúng tôi giải lại 01–03 với seed 0 trên một máy thứ hai (Apple M5, cùng phiên bản JuMP và HiGHS) và kiểm tra nghiệm bằng script `verify_solutions.jl`. Nghiệm của setA-01 và 02 có chứa chu trình rời (6 và 24 segment), nghiệm của setA-03 thì không. Sau khi bỏ các chu trình này, mọi segment path đọc ra đều đi từ nguồn tới đích với tối đa $maxSeg$ segment, và MLU tính lại từ các path đúng bằng U. Số thay đổi thật nằm trong budget: $dist(P^0, P^1) = 0$ ở 01 và 02 (solver giữ nguyên routing ở cả hai chu kỳ) và bằng $8 \leq \kappa(1) = 53$ ở 03 (hai demand đổi đường). Ở lần chạy kiểm tra này, setA-03 chưa chứng minh được tối ưu trong giới hạn thời gian, nhưng nghiệm tìm được có đúng giá trị 0.9435 đã được chứng minh tối ưu ở Bảng 5. Vậy các giá trị tối ưu trong Bảng 5 đều đạt được bởi nghiệm hợp lệ của bài toán T-ASR.

4 Kết luận và hướng phát triển

Chúng tôi đã cài đặt mô hình MILP của đề cho bài toán T-ASR bằng Julia/JuMP với solver HiGHS, gồm:

- phần đọc dữ liệu;
- tính hệ số chia luồng ECMP;
- mô hình một chu kỳ;
- mô hình hai chu kỳ có budget đã tuyến tính hoá.

Trên instance toy, mô hình cho đúng giá trị tối ưu, được xác nhận bằng vét cạn.

Trên SetA, mô hình một chu kỳ giải tối ưu bốn instance (01, 02, 03, 05), với MLU giảm từ 5.6% tới 65.3% so với định tuyến theo đường ngắn nhất. Mô hình hai chu kỳ giải tối ưu ba instance (01–03), và kiểm tra lại cho thấy các giá trị tối ưu này đạt được bởi nghiệm hợp lệ, dù solver có trả về chu trình rời. Ở cả hai bước, cây branch-and-bound gần như chỉ có nút gốc. Giới hạn thực tế là bộ nhớ và thời gian giải LP gốc của các mô hình lớn. Trong quá trình cài đặt, có ba điểm cần cẩn thận:

- so sánh độ dài đường đi khi trọng số là số thực phải có sai số;
- mô hình (1)–(4) cho phép chu trình rời, nên cần kiểm tra lại nghiệm khi có budget;
- mục tiêu $\min U$ không cải thiện các chu kỳ không phải nút cổ chai.

Hạn chế chính là kích thước: số biến tăng theo $|D| \cdot n^2$, nên mô hình đầy đủ chỉ dựng được cho các instance nhỏ. Có ba hướng phát triển tự nhiên:

- tiền xử lý để loại các segment path bị trội [8], áp dụng được cho một chu kỳ;
- sinh cột (column generation) trên đồ thị segment, với bài toán con là tìm đường đi có tối đa $maxSeg$ cung [7]; Jaumard et al. đã áp dụng cách này cho toàn bộ SetA, với gap LP–ILP trung bình 3.12% tại $t = 0$ và 13.05% tại $t = 1$ [3];
- với mục tiêu lexicographic đầy đủ, dùng chuỗi MILP như STELA hoặc ALEXA [2].

Tài liệu

[1] A. Benhamiche, Y. Carlinet, M. Chopin, E. Gourdin, N. Perrot. *Challenge EURO/ROADEF 2026 – Problem Description "Keep The Flow!"*. Orange Research, 2026.

[2] A. Benhamiche, K. Bouaachra, Y. Carlinet, M. Chopin, E. Gourdin, N. Perrot. On the T-Adaptive Segment Routing Problem. arXiv:2609.32896, 2026.

[3] B. Jaumard, N. P. Tran, J. M. Ziazet. Segment Routing Traffic Engineering with Time-Based Reconfiguration Constraints. arXiv:2610.04759, 2026.

[4] C. Filsfils et al. Segment Routing Architecture. RFC 8402, 2018.

[5] B. Fortz, M. Thorup. Internet traffic engineering by optimizing OSPF weights. IEEE INFOCOM, 2000.

[6] C. Bazgan, M. Chopin, A. Nichterlein, C. Richer. Parameterized complexity of segment routing. IEEE INFOCOM, 2025.

[7] M. Jadin, F. Aubry, P. Schaus, O. Bonaventure. CG4SR: Near optimal traffic engineering for segment routing with column generation. IEEE INFOCOM, 2019.

[8] H. Callebaut, J. De Boeck, B. Fortz. Preprocessing for segment routing optimization. *Networks* 82(4):459–478, 2023.